



EC200U&EG91xU Serie QuecOpen(SDK) **SSL Development Guide**

LTE Standard Module Series

Version: 1.1

Date: 2023-12-25

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-02-24	Kruskal ZHU	Creation of the document
1.0	2022-03-28	Kruskal ZHU/ Charis JIANG	First official release
1.1	2023-12-25	Kruskal ZHU	- Updated the document name based on the unified QuecOpen naming. - Added applicable modules EG912U-GL and EG915U series. - Updated enumeration ql_ssl_config_type_e of SSL parameter configuration types (Chapter 3.3.2.1). - Updated enumeration ql_ssl_authmode_e of SSL verification methods (Chapter 3.3.2.4). - Added enumeration ql_ssl_dtls_version_enum of DTLS versions (Chapter 3.3.2.5). - Added enumeration ql_ssl_renegotiation_type_e of whether to enable session renegotiation feature (Chapter 3.3.2.6). - Updated parameters in ql_ssl_config (Chapter 3.3.3.1).

Contents

About the Document.....	3
Contents	4
Table Index.....	6
1 Introduction	7
2 SSL API Calling Process	8
3 SSL API	10
3.1. Header File	10
3.2. API Overview.....	10
3.3. API Description.....	11
3.3.1. ql_ssl_conf_init.....	11
3.3.1.1. ql_ssl_error_code_e	11
3.3.2. ql_ssl_conf_set	13
3.3.2.1. ql_ssl_config_type_e	13
3.3.2.2. ql_ssl_version_type_e	15
3.3.2.3. ql_ssl_transport_type_e	15
3.3.2.4. ql_ssl_authmode_e	16
3.3.2.5. ql_ssl_dtls_version_enum	17
3.3.2.6. ql_ssl_renegotiation_type_e.....	17
3.3.3. ql_ssl_conf_get	17
3.3.3.1. ql_ssl_config.....	18
3.3.3.2. ql_ssl_handshake_timeout_cb.....	20
3.3.3.3. ql_ssl_session	20
3.3.4. ql_ssl_conf_set_by_id.....	21
3.3.5. ql_ssl_conf_get_by_id.....	22
3.3.6. ql_ssl_conf_free	22
3.3.7. ql_ssl_new.....	22
3.3.7.1. ql_ssl_context.....	23
3.3.8. ql_ssl_setup	23
3.3.9. ql_ssl_set_socket_fd.....	24
3.3.10. ql_ssl_set_hostname	24
3.3.11. ql_ssl_handshake	24
3.3.12. ql_ssl_close_notify	25
3.3.13. ql_ssl_get_bytes_avail	25
3.3.14. ql_ssl_read	26
3.3.15. ql_ssl_write.....	26
3.3.16. ql_ssl_free	27
3.3.17. ql_ssl_handshake_finished.....	27
3.3.18. ql_ssl_ciphersuit_is_valid	28
4 Example	29

5 Appendix References 30

Table Index

Table 1: API Overview 10

Table 2: Related Documents 30

Table 3: Terms and Abbreviations 30

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on SDK build environment. It introduces SSL API, its calling process and examples for Quectel EC200U series and EG91xU family modules.

2 SSL API Calling Process

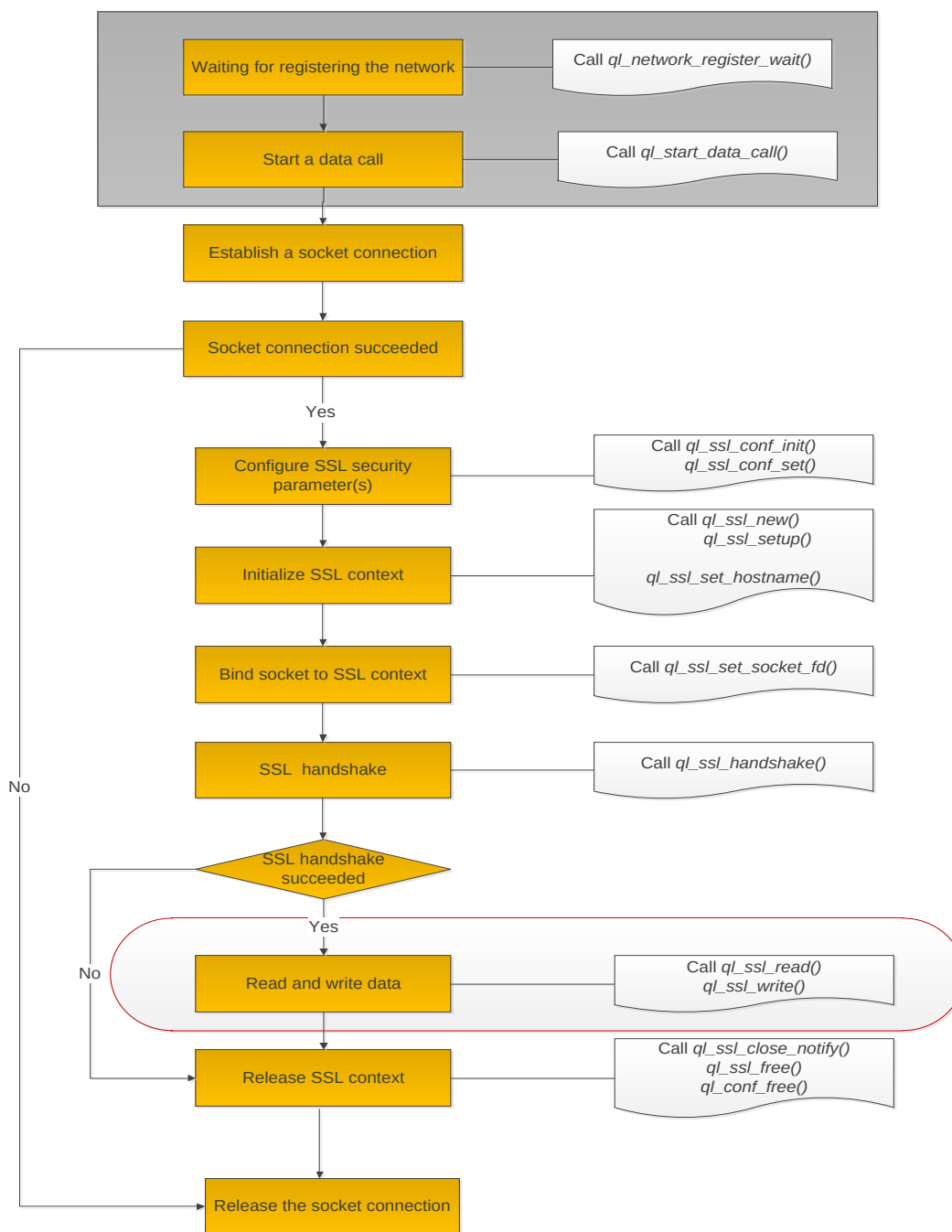


Figure 1: SSL API Calling Process

Before establishing an SSL session connection, you need to register the network and start a data call to establish a data channel (as shown in the gray part of the figure above), see **document [2]** for details. If the data channel has been established in other tasks, you can skip this step.

SSL session is established based on the socket connection, so you need to create a socket, bind the activated data channel IP and establish a socket connection before the SSL handshake. Once the socket connection is established successfully, you need to configure the SSL security parameters, initialize SSL context and bind the corresponding socket descriptor to the specified SSL context. After a successful SSL handshake, the data can be read and written. The SSL context and socket connection can be released once the data is read and written.

3 SSL API

3.1. Header File

ql_ssl.h, the header file of SSL API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

3.2. API Overview

Table 1: API Overview

Function	Description
<i>ql_ssl_conf_init()</i>	Initializes SSL default configuration parameters.
<i>ql_ssl_conf_set()</i>	Sets the specified SSL configuration parameters.
<i>ql_ssl_conf_get()</i>	Gets the specified SSL configuration parameters.
<i>ql_ssl_conf_set_by_id()</i>	Sets SSL context configuration parameters corresponding to SSL context ID.
<i>ql_ssl_conf_get_by_id()</i>	Gets SSL context configuration parameters corresponding to SSL context ID.
<i>ql_ssl_conf_free()</i>	Releases the resources occupied by SSL configuration parameters.
<i>ql_ssl_new()</i>	Generates a new SSL context.
<i>ql_ssl_setup()</i>	Sets an SSL context.
<i>ql_ssl_set_socket_fd()</i>	Binds socket descriptor and SSL context.
<i>ql_ssl_set_hostname()</i>	Sets the server domain name and is only valid when SNI is set.
<i>ql_ssl_handshake()</i>	Performs the SSL handshake.

<code>ql_ssl_close_notify()</code>	Notifies the server that the SSL connection is closing.
<code>ql_ssl_get_bytes_avail()</code>	Gets the length of readable data from the data buffer in SSL context.
<code>ql_ssl_read()</code>	Reads the readable data from the data buffer in SSL context.
<code>ql_ssl_write()</code>	Sends data to the server through SSL context.
<code>ql_ssl_free()</code>	Releases the resources occupied by the SSL context.
<code>ql_ssl_handshake_finished()</code>	Queries whether SSL handshake has finished.
<code>ql_ssl_ciphersuit_is_valid()</code>	Queries whether the cipher suit ID is valid.

3.3. API Description

3.3.1. ql_ssl_conf_init

This function initializes SSL default configuration parameters.

- **Prototype**

```
int ql_ssl_conf_init(ql_ssl_config *conf)
```

- **Parameter**

conf:

[In] SSL configuration handle. See **Chapter 3.3.3.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.1.1. ql_ssl_error_code_e

The SSL result codes indicate whether the function is executed successfully or not. The enumeration of SSL result codes:

```
typedef enum{
    QL_SSL_SUCCESS                = 0,
    QL_SSL_ERROR_UNKOWN          = -1,
```

```

QL_SSL_ERROR_WOUNDBLOCK      = -2,
QL_SSL_ERROR_INVALID_PARAM   = -3,
QL_SSL_ERROR_OUT_OF_MEM      = -4,
QL_SSL_ERROR_NOT_SUPPORT     = -5,
QL_SSL_ERROR_HS_FAILURE      = -6,
QL_SSL_ERROR_DECRYPT_FAILURE  = -7,
QL_SSL_ERROR_ENCRYPT_FAILURE  = -8,
QL_SSL_ERROR_HS_INPROGRESS   = -9,
QL_SSL_ERROR_BAD_REQUEST     = -10,
QL_SSL_ERROR_WANT_READ       = -11,
QL_SSL_ERROR_WANT_WRITE      = -12,
QL_SSL_ERROR_SOCKET_RESET    = -13,
}ql_ssl_error_code_e

```

● Member

Member	Description
<i>QL_SSL_SUCCESS</i>	Successful execution.
<i>QL_SSL_ERROR_UNKOWN</i>	Unknown error.
<i>QL_SSL_ERROR_WOUNDBLOCK</i>	The operation is not completed and awaits an asynchronous notification for its result.
<i>QL_SSL_ERROR_INVALID_PARAM</i>	Invalid parameter(s).
<i>QL_SSL_ERROR_OUT_OF_MEM</i>	Out of memory.
<i>QL_SSL_ERROR_NOT_SUPPORT</i>	The operation is not supported.
<i>QL_SSL_ERROR_HS_FAILURE</i>	SSL handshake failed.
<i>QL_SSL_ERROR_DECRYPT_FAILURE</i>	Failed to decrypt.
<i>QL_SSL_ERROR_ENCRYPT_FAILURE</i>	Failed to encrypt.
<i>QL_SSL_ERROR_HS_INPROGRESS</i>	SSL handshake in progress.
<i>QL_SSL_ERROR_BAD_REQUEST</i>	Request error.
<i>QL_SSL_ERROR_WANT_READ</i>	No readable data.
<i>QL_SSL_ERROR_WANT_WRITE</i>	No writable data.
<i>QL_SSL_ERROR_SOCKET_RESET</i>	Socket disconnected abnormally.

3.3.2. ql_ssl_conf_set

This function sets the specified SSL configuration parameters.

● Prototype

```
int ql_ssl_conf_set(ql_ssl_config *conf, int type, ...)
```

● Parameter

conf:

[In] SSL configuration handle. See **Chapter 3.3.3.1** for details.

type:

[In] SSL parameter configuration type. See **Chapter 3.3.2.1** for details.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.2.1. ql_ssl_config_type_e

The enumeration of SSL parameter configuration types:

```
typedef enum{
    QL_SSL_CONF_VERSION          = 1,
    QL_SSL_CONF_TRANSPORT        = 2,
    QL_SSL_CONF_CIPHERSUITE      = 3,
    QL_SSL_CONF_AUTHMODE         = 4,
    QL_SSL_CONF_CACERT           = 5,
    QL_SSL_CONF_OWNCERT          = 6,
    QL_SSL_CONF_SNI              = 7,
    QL_SSL_CONF_HS_TIMEOUT       = 8,
    QL_SSL_CONF_IGNORE_LOCALTM   = 9,
    QL_SSL_CONF_HS_TIMEOUT_FUNC  = 10,
    QL_SSL_CONF_IGNORE_INVALID_CERT_SIGN = 11,
    QL_SSL_CONF_IGNORE_CERT_ITEM = 12,
    QL_SSL_CONF_IGNORE_MULTI_CERTCHAIN_VERIFY = 13
#ifdef QL_SSL_TLS_SESSION_CACHE_FEATURE
    QL_SSL_CONF_SESSION_CACHE   = 14,
#endif
    QL_SSL_CONF_CACERT_BUFFER    = 15,
    QL_SSL_CONF_OWNCERT_BUFFER   = 16,
#ifdef QL_SSL_DTLS_FEATURE
```

```

        QL_SSL_CONF_DTLSVERSION  = 17,
    #endif

    #ifdef QL_SSL_PSK_FEATURE
        QL_SSL_CONF_PSK_IDENTITY  = 18,
        QL_SSL_CONF_PSK_KEY       = 19,
    #endif

    #ifdef QL_SSL_RENEGOTIATION_FEATURE
        QL_SSL_CONF_RENEGOTIATION = 20,
    #endif
}ql_ssl_config_type_e

```

● Member

Member	Description
<i>QL_SSL_CONF_VERSION</i>	Sets SSL version. See Chapter 3.3.2.2 for details.
<i>QL_SSL_CONF_TRANSPORT</i>	Sets SSL communication method. See Chapter 3.3.2.3 for details.
<i>QL_SSL_CONF_CIPHERSUITE</i>	Sets SSL cipher suite.
<i>QL_SSL_CONF_AUTHMODE</i>	Set SSL verification method. See Chapter 3.3.2.4 for details.
<i>QL_SSL_CONF_CACERT</i>	Sets SSL CA certificate list.
<i>QL_SSL_CONF_OWNCERT</i>	Sets SSL local certificate.
<i>QL_SSL_CONF_SNI</i>	Sets whether to enable SSL SNI.
<i>QL_SSL_CONF_HS_TIMEOUT</i>	Sets the SSL handshake timeout.
<i>QL_SSL_CONF_IGNORE_LOCALTM</i>	Sets whether to ignore local time.
<i>QL_SSL_CONF_HS_TIMEOUT_FUNC</i>	Sets the timeout manipulation function of SSL handshake.
<i>QL_SSL_CONF_IGNORE_INVALID_CERT_SIGN</i>	Sets whether to ignore the check of SSL certificate sent by the server.
<i>QL_SSL_CONF_IGNORE_CERT_ITEM</i>	Sets whether to ignore the check of SSL certificate items sent by the server.
<i>QL_SSL_CONF_IGNORE_MULTI_CERTCHAIN_VERIFY</i>	Sets whether to ignore SSL multi-level certificate chain verification.
<i>QL_SSL_CONF_SESSION_CACHE</i>	Sets whether to enable SSL session reuse function.
<i>QL_SSL_CONF_CACERT_BUFFER</i>	Sets the buffer mode of SSL CA certificate.

<code>QL_SSL_CONF_OWNCERT_BUFFER</code>	Sets the buffer mode of SSL local certificate.
<code>QL_SSL_CONF_DTLSVERSION</code>	Sets the DTLS version. See Chapter 3.3.2.5 for details.
<code>QL_SSL_CONF_PSK_IDENTITY</code>	Set the pre-shared key identifier.
<code>QL_SSL_CONF_PSK_KEY</code>	Set the pre-shared key to be used.
<code>QL_SSL_CONF_RENEGOTIATION</code>	Sets whether to enable session renegotiation feature. See Chapter 3.3.2.6 for details.

3.3.2.2. ql_ssl_version_type_e

The enumeration of SSL versions:

```
typedef enum
{
    QL_SSL_VERSION_0 = 0,
    QL_SSL_VERSION_1,
    QL_SSL_VERSION_2,
    QL_SSL_VERSION_3,
    QL_SSL_VERSION_ALL
} ql_ssl_version_type_e
```

● Member

Member	Description
<code>QL_SSL_VERSION_0</code>	SSL 3.0 version
<code>QL_SSL_VERSION_1</code>	TLS 1.0 version (SSL 3.1)
<code>QL_SSL_VERSION_2</code>	TLS 1.1 version (SSL 3.2)
<code>QL_SSL_VERSION_3</code>	TLS 1.2 version (SSL 3.3)
<code>QL_SSL_VERSION_ALL</code>	All SSL versions

3.3.2.3. ql_ssl_transport_type_e

The enumeration of SSL communication methods:


```
typedef enum{
    QL_SSL_TLS_PROTOCOL = 0,
    QL_SSL_DTLS_PROTOCOL = 1,
}ql_ssl_transport_type_e
```

● Member

Member	Description
<i>QL_SSL_TLS_PROTOCOL</i>	TLS, based on TCP socket communication.
<i>QL_SSL_DTLS_PROTOCOL</i>	DTLS, based on UDP socket communication.

3.3.2.4. ql_ssl_authmode_e

The enumeration of SSL verification methods:

```
typedef enum
{
    QL_SSL_VERIFY_NULL          = 0x0000,
    QL_SSL_VERIFY_SERVER        = 0x0001,
    QL_SSL_VERIFY_CLIENT_SERVER = 0x0002,
#ifdef QL_SSL_PSK_FEATURE
    QL_SSL_VERIFY_PSK           = 0x0003,
#endif
} ql_ssl_authmode_e
```

● Member

Member	Description
<i>QL_SSL_VERIFY_NULL</i>	When the server does not require client verification, this option indicates that the client does not verify the server. When the server requires client verification, setting this option causes SSL handshake failure.
<i>QL_SSL_VERIFY_SERVER</i>	When the server does not require client verification, this option indicates that the client needs to verify the server. When the server requires client verification, setting this option causes SSL handshake failure.
<i>QL_SSL_VERIFY_CLIENT_SERVER</i>	When the server does not require client verification, this option is equivalent to <i>QL_SSSL_VERIFY_SERVER</i> . When the server requires client verification, this option must be set.

`QL_SSL_VERIFY_PSK`

Set the SSL verification mode to pre-shared key. When setting this method, please call `ql_ssl_conf_set()` to configure `QL_SSL_CONF_PSK_IDENTITY` and `QL_SSL_CONF_PSK_KEY`.

3.3.2.5. ql_ssl_dtls_version_enum

The enumeration of DTLS versions:

```
typedef enum{
    QL_SSL_VER_DTLS10  = 0,    /**< DTLS protocol ver. 1.0. > */
    QL_SSL_VER_DTLS12  = 1,    /**< DTLS protocol ver. 1.2. > */
}ql_ssl_dtls_version_enum
```

● Member

Member	Description
<code>QL_SSL_VER_DTLS10</code>	DTLS V1.0
<code>QL_SSL_VER_DTLS12</code>	DTLS V1.2

3.3.2.6. ql_ssl_renegotiation_type_e

The enumeration of whether to enable session renegotiation feature:

```
typedef enum{
    QL_SSL_RENEGOTIATION_DISABLED  = 0,
    QL_SSL_RENEGOTIATION_ENABLED   = 1,
}ql_ssl_renegotiation_type_e
```

● Member

Member	Description
<code>QL_SSL_RENEGOTIATION_DISABLED</code>	Disable session renegotiation feature
<code>QL_SSL_RENEGOTIATION_ENABLED</code>	Enable session renegotiation feature

3.3.3. ql_ssl_conf_get

This function gets the specified SSL configuration parameters.

● Prototype

```
int ql_ssl_conf_get(ql_ssl_config *conf, int type, ...)
```

● Parameter

conf:

[In] SSL attribute configuration. See **Chapter 3.3.3.1** for details.

type:

[In] SSL parameter configuration type. See **Chapter 3.3.2.1** for details.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.3.1. ql_ssl_config

The structure of SSL attribute configurations:

```
typedef struct{
    int                ssl_version;
    int                transport;
    int                *ciphersuites;
    int                auth_mode;
    int                sni_enable;
    char               *ca_cert_path[QL_MAX_CA_CERT_CNT];
    char               *own_cert_path;
    char               *own_key_path;
    char               *own_key_pwd;
    char               *ca_cert_buffer[QL_MAX_CA_CERT_CNT];
    int                ssl_negotiate_timeout;
    ql_ssl_handshake_timeout_cb negotiate_timeout_cb;
    void               *negotiate_timeout_cb_arg;
    int                ignore_invalid_certsign;
    uint32_t           ignore_certitem;
    int                ignore_multi_certchain_verify;
    bool               client_cert_type;
#ifdef QL_SSL_TLS_SESSION_CACHE_FEATURE
    ql_ssl_session     ssl_session_cache;
#endif
#ifdef QL_SSL_DTLS_FEATURE
    int                dtls_version;
#endif
}
```

```

#ifdef QL_SSL_PSK_FEATURE
char                                *psk_identity;
char                                *psk_key;
#endif

#ifdef QL_SSL_RENEGOTIATION_FEATURE
int                                renegotiation;
#endif
}ql_ssl_config

```

● Parameter

Type	Parameter	Description
int	<i>ssl_version</i>	SSL version.
int	<i>transport</i>	SSL protocol type. 0 TLS 1 DTLS
int	<i>ciphersuites</i>	Cipher suite.
int	<i>auth_mode</i>	Verification method.
int	<i>sni_enable</i>	Whether to enable SNI.
char*	<i>ca_cert_path</i>	CA certificate list. It can be up to <i>QL_MAX_CA_CERT_CNT</i> .
char*	<i>own_cert_path</i>	Local certificate path.
char*	<i>own_key_path</i>	Local key file path.
char*	<i>own_key_pwd</i>	Encrypted password for local key file, NULL if there is no encrypted password.
char*	<i>ca_cert_buffer</i>	Buffer content of CA certificate.
int	<i>ssl_negotiate_timeout</i>	The maximum timeout of SSL negotiation.
<i>ql_ssl_handshake_timeout_cb</i>	<i>negotiate_timeout_cb</i>	SSL handshake timeout callback function. See Chapter 3.3.3.2 for details.
void	<i>negotiate_timeout_cb_arg</i>	Parameters passed into the SSL handshake timeout callback function.
int	<i>ignore_invalid_certsign</i>	Set whether to ignore the check of the SSL certificate sent by the server.
uint32_t	<i>ignore_certitem</i>	Set whether to ignore the check of SSL certificate items sent by the server.
int	<i>ignore_multi_certchain_verify</i>	Set whether to ignore verification of multi-level

		certificate chains.
bool	<i>client_cert_type</i>	CA certificate type. 0 QL_SSL_CLIENT_CERT_FILE 1 QL_SSL_CLIENT_CERT_BUFFER
<i>ql_ssl_session</i>	<i>ssl_session_cache</i>	SSL session reuse configuration. See Chapter 3.3.3.3 for details.
int	<i>dtls_version</i>	DTLS version type. 0 QL_SSL_VER_DTLS10 1 QL_SSL_VER_DTLS12
char*	<i>psk_identity</i>	Set the pre-shared key identifier.
char*	<i>psk_key</i>	Set the pre-shared key to be used.
int	<i>renegotiation</i>	Set whether to enable session renegotiation feature. 0 QL_SSL_RENEGOTIATION_DISABLED 1 QL_SSL_RENEGOTIATION_ENABLED

3.3.3.2. ql_ssl_handshake_timeout_cb

This callback function defines the function pointer of the SSL handshake timeout callback function.

● Prototype

```
typedef void(*ql_ssl_handshake_timeout_cb)(ql_ssl_context *ssl_ctx, void *arg)
```

● Parameter

ssl_ctx:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

arg:

[In] Pointer to the callback function parameters customized by users.

3.3.3.3. ql_ssl_session

The structure of SSL session reuse configurations:

```
typedef struct {
    uint8_t          session_cache_enable;
    ip_addr_t        remote_ip;
    uint16_t         remote_port;
```

```
uint8_t      hostname_temp[256];
uint8_t      session_hostname[256];
mbedtls_ssl_session ssl_session;
} ql_ssl_session
```

● Parameter

Type	Parameter	Description
uint8_t	<i>session_cache_enable</i>	Whether to enable session reuse. 0 Disable 1 Enable
<i>ip_addr_t</i>	<i>remote_ip</i>	Saves the IP address of the peer server.
uint16_t	<i>remote_port</i>	Saves the port number of the peer server.
uint8_t	<i>hostname_temp</i>	Stores the host name of the peer server temporarily.
uint8_t	<i>session_hostname</i>	Real host name used for session reuse.
<i>mbedtls_ssl_session</i>	<i>ssl_session</i>	Stores the current session data.

3.3.4. ql_ssl_conf_set_by_id

This function sets SSL context configuration parameter corresponding to SSL context ID.

● Prototype

```
int ql_ssl_conf_set_by_id(int ctx_id, int type, ...)
```

● Parameter

ctx_id:

[In] Integer type. SSL context ID. Range: 0–5.

type:

[In] SSL parameter configuration type. See **Chapter 3.3.2.1** for details.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.5. ql_ssl_conf_get_by_id

This function gets SSL context configuration parameter corresponding to SSL context ID.

- **Prototype**

```
int ql_ssl_conf_get_by_id(int ctx_id, int type, ...)
```

- **Parameter**

ctx_id:

[In] Integer type. SSL context ID. Range: 0–5.

type:

[In] SSL parameter configuration type. See **Chapter 3.3.2.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.6. ql_ssl_conf_free

This function releases the resources occupied by SSL configuration parameters.

- **Prototype**

```
int ql_ssl_conf_free(ql_ssl_config *conf)
```

- **Parameter**

conf:

[In] SSL configuration handle. See **Chapter 3.3.3.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.7. ql_ssl_new

This function generates a new SSL context.

- **Prototype**

```
int ql_ssl_new(ql_ssl_context *ssl)
```

- **Parameter**

ssl:

[Out] SSL context handle. See **Chapter 3.3.7.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.7.1. ql_ssl_context

The SSL context is defined as follows:

```
typedef int ql_ssl_context
```

- **Parameter**

Type	Parameter	Description
int	<i>ql_ssl_context</i>	SSL context handle.

3.3.8. ql_ssl_setup

This function sets an SSL context.

- **Prototype**

```
int ql_ssl_setup(ql_ssl_context *ssl, ql_ssl_config *conf)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

conf:

[In] SSL attribute configuration. See **Chapter 3.3.3.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.9. ql_ssl_set_socket_fd

This function binds the socket descriptor and SSL context.

- **Prototype**

```
int ql_ssl_set_socket_fd(ql_ssl_context *ssl, int sock_fd)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

sock_fd:

[In] Socket descriptor.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.10. ql_ssl_set_hostname

This function sets the server domain name and is only valid when the SNI is set.

- **Prototype**

```
int ql_ssl_set_hostname(ql_ssl_context *ssl, const char *hostname)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

hostname:

[In] Server domain name.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.11. ql_ssl_handshake

This function performs the SSL handshake.

- **Prototype**

```
int ql_ssl_handshark(ql_ssl_context *ssl)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.12. ql_ssl_close_notify

This function notifies the server that the SSL connection is closing.

- **Prototype**

```
int ql_ssl_close_notify(ql_ssl_context *ssl)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.13. ql_ssl_get_bytes_avail

This function gets the length of readable data from the data buffer in SSL context once the SSL handshake is completed.

- **Prototype**

```
int ql_ssl_get_bytes_avail(ql_ssl_context *ssl)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

- **Return Value**

Less than 0 Returns SSL result codes. See **Chapter 3.3.1.1** for details.
 Greater than 0 Returns the length of the readable data in the buffer.

3.3.14. ql_ssl_read

This function reads the readable data from the data buffer in SSL context once the SSL handshake is completed.

- **Prototype**

```
int ql_ssl_read(ql_ssl_context *ssl, unsigned char *buf, size_t len)
```

- **Parameter**

ssl:
 [In] SSL context handle. See **Chapter 3.3.7.1** for details.

buf:
 [In] Buffer for storing readable data.

len:
 [In] Length of data to be read.

- **Return Value**

Less than 0 Returns SSL result codes. See **Chapter 3.3.1.1** for details.
 Not less than 0 Returns the number of data bytes successfully read from the buffer.

3.3.15. ql_ssl_write

This function sends data to server through SSL context once the SSL handshake has finished.

- **Prototype**

```
int ql_ssl_write(ql_ssl_context *ssl, const unsigned char *buf, size_t len)
```

- **Parameter**

ssl:
 [In] SSL context handle. See **Chapter 3.3.7.1** for details.

buf:
 [In] Buffer for storing data.

len:

[In] Length of data to be sent.

- **Return Value**

Less than 0 Returns SSL result codes. See **Chapter 3.3.1.1** for details.

Not less than 0 Returns the number of data bytes successfully read from the buffer.

3.3.16. ql_ssl_free

This function releases the resources occupied by the SSL context.

- **Prototype**

```
int ql_ssl_free(ql_ssl_context *ssl)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.17. ql_ssl_handshake_finished

This function queries whether the SSL handshake has finished.

- **Prototype**

```
int ql_ssl_handshark_finished(ql_ssl_context *ssl)
```

- **Parameter**

ssl:

[In] SSL context handle. See **Chapter 3.3.7.1** for details.

- **Return Value**

1 SSL handshake process has completed.

0 SSL handshake process has not completed.

Less than 0 Returns SSL result codes. See **Chapter 3.3.1.1** for details.

3.3.18. ql_ssl_ciphersuit_is_valid

This function queries whether the specified cipher suite ID is valid.

- **Prototype**

```
int ql_ssl_ciphersuit_is_valid(int cs_id)
```

- **Parameter**

cs_id:

[In] Cipher suite ID. See *ql_ssl.h* in SDK for details.

- **Return Value**

- 1 Valid
- 0 Invalid

4 Example

ssl_demo.c, the example file of SSL API, is in the `\components\ql-application\ssl` QuecOpen SDK directory. The example file covers various operations such as establishing a TCP connection, configuring SSL session attributes, establishing an SSL connection, reading and writing data through the SSL connection and other operations. You can view the complete SSL API example in this directory.

5 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen_Data_Call_API_Reference_Manual

Table 3: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
AP	Application Processor
App	Application
DTLS	Datagram Transport Layer Security
EVB	Evaluation Board
IoT	Internet of Things
PC	Personal Computer
RTOS	Real-Time Operating System
SDK	Software Development Kit
SSL	Secure Sockets Layer
SNI	Server Name Indication
TLS	Transport Layer Security
USB	Universal Serial Bus

UDPUser Datagram Protocol
